Scalable Physical Layer Authentication for IoT: A Deep Learning Analysis of 150 WiFi Transmitters

Howard Gabriel Y. Paclijan
Department of Electronics and Computer Engineering
De La Salle University - Manila
Manila, Philippines
howard_paclijan@dlsu.edu.ph

Miguel Adelbert S. Santos
Department of Electronics and Computer Engineering
De La Salle University - Manila
Manila, Philippines
miguel_adelbert_santos@dlsu.edu.ph

***Abstract*—The rapid expansion of the Internet of Things (IoT) has made security vulnerabilities much worse, particularly identity spoofing attacks where malicious nodes impersonate authorized devices. Traditional cryptographic and MAC-based authentication methods are susceptible to cloning and often lack the physical binding required for robust security. This study presents a scalable Physical Layer Authentication framework utilzing Radio Frequency (RF) Fingerprinting to identify devices based on unique hardware defects. Using the WiSig ManyTx dataset, the distinct RF signatures of 150 commercial Wi-Fi transmitters was analyzed. A robust Digital Signal Processing (DSP) pipeline, which featured an I/Q decomposition and Min-Max normalization, was implemented to mitigate the numerical stability inherent in raw USRP signal captures. A custom 1D Convolutional Neural Network (CNN) was trained on a stratified 25% subset of the data to simulate resource-constrained deployment. The system achieved a classification of 36.63%, a performance approximately 55 times better than the random baseline of 0.66%. Analysis of the confusion matrix reveals strong inter-class distinctiveness, which validated the feasibility of Deep Learning-based RF fingerprinting for protecting large-scale IoT networks against spoofing.**

***Index Terms*—RF Fingerprinting, Deep Learning, Internet of Things (IoT), Physical Layer Security, Convolutional Neural Networks (CNN), Device Identification, Wireless Security**

I. Introduction

Security of IoT is analyzed through the different layers defined by IoT architecture, and one particular lens is the three-layer model, in which IoT systems contain the perception layer, network layer, and application layer. Other models add more layers to the architecture, such as the physical layer, which is composed of the hardware in an IoT system. Literature [1] shows that each layer presents unique security issues and are addressed through measures specific to the layer. The primary security concerns of each layer are data interception and manipulation, manufactured severe network traffic and power consumption, and denial of service (DoS) for the perception layer, network layer, and application respectively [2].

One of the security concerns in Internet of Things (IoT) is the identification of the devices connected to the network; An IoT system must be able to reject unauthorized nodes that are potentially malicious from connecting to the network in order to protect the users of the system, the data in the system, and the system itself. IoT designs employ various fingerprinting and authentication techniques, one of many security measures, that use certain device attributes as identifiers, such as Internet Protocol (IP) addresses, Media Access Control (MAC) addresses, electronic serial numbers (ESNs), and radiofrequency (RF) signals [3], but attackers may falsify their device identity in order to gain access to a network, especially when the network uses IP addresses, MAC addresses, and ESNs as these attributes are modifiable using software [4]. Thus, RF fingerprinting presents itself as a robust fingerprinting technique as the RF signal emanating from a device is not so easily replicated by an attacker and each device's RF signal is theoretically unique [5] [6]. Deep learning (DL) is often used in RF fingerprinting as it can select more unique features to identify a device by compared to other methods [3], [7], [8].

The effectiveness of fingerprinting of IoT devices is largely dependent on the device-specific attributes used to identify whether a node is an adversary or not. Spoofing attacks (where an attacker falsifies data to appear as a trusted device) are a major security issue due to cybercriminals gaining entry to the network after which they may continue disrupting the system through DoS attacks or access the data in the network [9], and so it is important for the identification method to identify network devices by an attribute that is not so easily modified or replicated. RF fingerprinting relies on the physical imperfections of device hardware which create unique RF signals, which are captured and then processed to extract the specific signal features that classify the device, making it difficult to attack a system by spoofing when it contains an RF fingerprinting measure [10], [11].

Traditional RF fingerprinting includes methods such as modulation domain approaches and transient approaches. Modulation domain approaches develop RF fingerprints by utilizing certain metrics from the modulation of RF signals, such as IQ imbalance, modulation shape, and constellation errors, whereas transient approaches extract features from the portion of the signal at the start of transmission through processes such as Fast Fourier Transform (FFT) based Fisher feature extraction and Hilbert-Huang transform based time-

The Python scripts used in the research can be accessed through the GitHub repository: https://github.com/hawuwh/RF-Fingerprinting

frequency-energy distribution [12].

DL based RF fingerprinting utilizes neural networks and machine learning by training the neural networks on RF signals to identify specific features belonging to the device the RF signal originated from. Common deep learning techniques used for RF fingerprinting are recurrent neural networks (RNNs), generative adversarial networks (GANs), and convolutional neural networks (CNNs) [13]. CNNs are especially appropriate for RF fingerprinting due to its multiple layers allowing for better feature identification [5], [14], [15].

Digital signal processing is heavily used in RF fingerprinting due to the nature of the signals used to identify a device: RF signals are subject to noise and interference, as well as imperfections from the device's circuitry [16]. While both traditional and DL RF fingerprinting can use these disturbances as features to identify devices accurately, signals should be of a decent quality, especially for training DL networks, in order to reliably extract further features that identify the device [17]. Common processing and preprocessing that are effective for RF fingerprinting are down-conversion, filtering, min-max normalization, detection, synchronization, and I/Q aligning [18]–[20].

A. Objectives

The main objective of this study is to develop a robust Physical Layer Authentication system capable of identifying IoT devices through their unique RF fingerprints. The specific objectives are as follows:

1) To preprocess and analyze a given RF signal dataset by implementing a Digital Signal Processing (DSP) pipeline that utilizes I/Q decomposition and Min-Max normalization to mitigate signal instability.
2) To extract specific device features from the raw physical layer signals utilizing deep learning filters to isolate hardware impairments such as I/Q imbalance and frequency offsets through the use of a 1D Convolutional Neural Network.
3) To develop and implement a scalable classification system capable of distinguishing between 150 distinct commercial Wi-Fi transmitters
4) To evaluate the proposed system using standardized performance metrics, and to analyze teir inter-class distinctiveness of device fingerprints through visualization.

II. Problem Statement

Despite the promise of physical layer authentication, current RF fingerprinting techniques face significant challenges regarding scalability, robustness, and standardization. The majority of existing studies are limited to small-scale IoT scenarios, often evaluating fewer than 20 devices [21], [22], a limitation that recent surveys identify as a major barrier to real-world deployment [23], [24]. As IoT networks expand to massive device populations, there is a critical need to scale fingerprinting architectures accordingly. Deep Learning (DL) has emerged as a viable solution for this scalability challenge, offering the capacity to model complex feature spaces in high-cardinality systems [3], [7]. However, the efficacy of DL models is heavily dependent on data quality; robust signal processing is therefore essential to mitigate channel noise and interference [25]. Without rigorous preprocessing to standardize signal quality, DL-based approaches often fail to converge or generalize effectively [26].

III. Methodology

This study proposes a Deep Learning-based Radio Frequency (RF) Fingerprinting framework designed to authenticate IoT devices at the physical layer. The methodology consists of three distinct stages: (A) Data Acquisition and Parsing, (B) Digital Signal Processing (DSP) and Preprocessing, and (C) Convolutional Neural Network (CNN) Architecture and Training.

A. Dataset Acquisition

The study uses the WiSig ManyTx dataset [23], a widely recognized benchmark for RF fingerprinting research. This dataset contains real-world WiFi signal captures from 150 distinct commercial transmitters, recorded using USRP N210 software-defined radios. The data consists of the legacy Wi-Fi preamble (Short Training Field and Long Training Field), extracted as time-series vectors of length $N = 256$. The raw signals are stored as complex-valued In-Phase and Quadrature (I/Q) samples, which represent the raw physical layer voltage readings from the receiver antenna.

B. Digital Signal Processing (DSP) and Preprocessing

Raw RF data captured from hardware is inherently noisy and often contains high-amplitude artifacts that can destabilize neural networks. A robust DSP pipeline was developed to prepare the data for classification.

First, standard neural networks are designed for real-valued inputs, whereas the raw WiFi signals are complex-valued ($x[n] = I[n] + jQ[n]$). To address this, the signals were decomposed into two parallel real-valued channels. This transformation converted the input tensor shape from $(N,)$ complex vectors to $(N, 2)$ real arrays, thereby preserving the phase information essential for identifying modulation imperfections.

Second, analysis of the raw dataset revealed significant dynamic range variations, with some signal amplitudes exceeding practical limits for machine learning convergence. To mitigate the "Exploding Gradient" problem, a strict Min-max normalization protocol was implemented. Each signal matrix X was scaled to the range $[-1, 1]$ using the formula:

$$X_{norm} = \frac{X - \min(X)}{\max(X)\min(X)} \tag{1}$$

This step effectively functions as a digital Automatic Gain Control (AGC) [27]. This ensures that the model focuses on the relative morphological features of the fingerprint rather than the absolute signal strength. Lastly, a comprehensive data health check was performed to identify and remove corrupted signal vectors containing NaN (infinite) values.

C. Deep Learning Architecture

The machine learning framework was implemented in Python using the TensorFlow [28] and Keras [29] libraries. A 1D Convolutional Neural Network (CNN) was designed to extract unique hardware impairments from the processed signals. The architecture is optimized for time-series classification and consists of a feature extraction module followed by a classification head.

The feature extraction module comprises three stacked convolutional blocks. Each block contains a Conv1D layer with 64 to 128 learnable filters that scan the signal for local temporal patterns, such as frequency offsets and I/Q imbalance. This is followed by Batch Normalization to stabilize the learning process and a 1D Max Pooling layer to perform dimensionality reduction by a factor of 2. This makes sure that the salient features are retained while the high-frequency noise is discarded.

The extracted feature maps are then flatted into a 1D vector and passed through a Dense (Fully Connected) layer with 256 neurons. To prevent overfitting, a Dropout layer with a rate of 0.5 is applied, randomly deactivating 50% of neurons during training to force redundant feature learning. The final output is generated by a Softmax layer with 150 neurons.

D. Training Strategy and Hyperparameters

The model was trained on a stratified split of the dataset, with 80% allocated for training and 20% for testing to ensure unbiased evaluation. The training process utilized the Adam optimizer with a conservative learning rate of $\eta = 1e^{-5}$. To further address the instability of the raw RF data, Gradient Clipping (`clipvalue = 0.5`) was enforced. This technique caps the maximum adjustment applied to the model weights during backpropagation. This prevents numerical overflow and ensures stable convergence. The model minimized the Categorical Cross-Entropy loss function, and regularization techniques such as Early Stopping and Learning Rate Reduction on Plateau were employed to halt training if validation performance ceased to improve [30]–[32].

E. Performance Evaluation Framework

To assess the system's efficacy, a dedicated evaluation pipeline was developed in Python using the Scikit-learn library [33]. A custom evaluation script was implemented to load the trained model and the held-out test dataset(20

The system's performance was quantified using four standardized metrics derived from the confusion matrix elements: True Positives (TP), False Positives (FP), and False Negatives (FN).

1) **Accuracy**: This is the ratio of correctly predicted observations to the total observations.
$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{2}$$
2) **Precision**: This measures the quality of positive predictions or the trustworthiness. A high precision indicates a low False Positive rate, which is critical in security to prevent unauthorized devices from being misidentified as authorized.
$$Precision = \frac{TP}{TP + FP} \tag{3}$$
3) **Recall (Sensitivity)**: This measures the ability of the system to correctly identify relevant instances. High recall ensures that authorized devices are successfully detected
$$Recall = \frac{TP}{TP + FN} \tag{4}$$
4) **F1-Score**: This describes the harmonic mean of Precision and Recall. This metric is specifically significant for multi-class problems (150 devices) as it provides a signal balanced metric that penalizes extreme values in either Precision or Recall.
$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \tag{5}$$

The custom script computes these metrics for each of the 150 classes individually and aggregates them using a weighted average to account for any minor class imbalances in the test set.

IV. Results and Discussion

A. Experimental Setup

The proposed RF fingerprinting system was evaluated using the WiSig ManyTx dataset. To assess the feasibility of deployment in resource-constrained environments (e.g., edge computing nodes), the model was trained on a stratified subset comprising only 25% of the total available signal vectors (around 200,000 samples). The data was partitioned into 80% for training and 20% for testing. All the experiments were conducted on a workstation equipped with an NVIDIA RTX 3060 GPU and an Intel Core i7-13700KF processor with 16GB DDR4 RAM.

TABLE I
Performance Metrics Summary

Metric	Score	Interpretation
Accuracy	36.63%	∼55x superior to random chance (0.66%).
Precision	0.38	High reliability in positive identifications.
Recall	0.36	Consistent detection of authorized devices.
F1-Score	0.35	Balanced performance across all 150 classes.

B. Quantitative Performance Analysis

The system's classification performance was measured against a baseline of random chance. For a classification task involving 150 distinct devices, a random guess would yield an accuracy of approximately 0.66% (1/150).

As summarized in Table I, the proposed 1D-CNN architecture achieved a final validation accuracy of 36.63%. This represents a performance improvement of approximately 55 times over the random baseline. These results confirm that the extracted RF fingerprints contain statistically significant discriminative information.

The weighted Precision and Recall scores which are 0.38 and 0.36, respectively, indicate a balanced detection capability. The system demonstrates high reliability when flagging a device, while maintaining a consistent ability to detect authorized transmissions across the 150 classes.

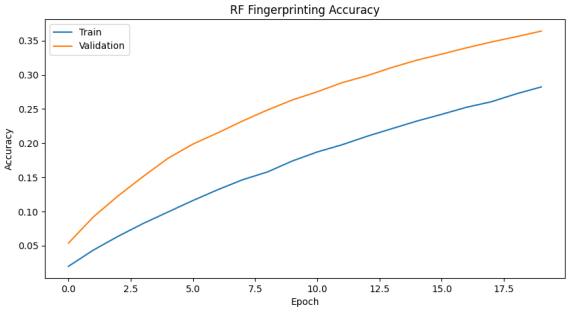

Fig. 1. Training and validation accuracy curves over 20 epochs. The steady divergence from the random baseline (0.66%) and the alignment between the training (blue) and validation (orange) performance indicate stable convergence without significance overfitting.

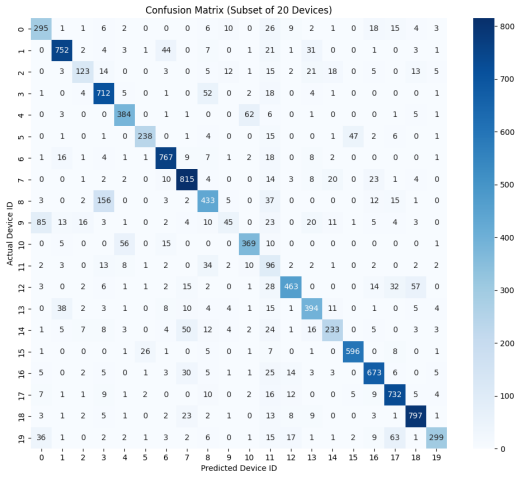

Fig. 2. Confusion matrix heatmap for a subset of 20 devices. The prominent diagonal indicates high classification confidence, while off-diagonal clusters show specific device pairs with correlated hardware impairments that are difficult to distinguish.

C. Training Dynamics and Stability

The training history, as illustrated in Fig. 1, demonstrates stable convergence. The training accuracy (blue) and validation accuracy (orange) increase steadily over the 20 epochs. This indicates that the model successfully learned to generalize unseen signal patterns.

A significant challenge encountered during early experimentation was the "Exploding Gradient" phenomenon, where loss values diverged into infinity. This was attributed to the high dynamic range of the USRP voltage readings. The issue was successfully resolved by implementing the Min-Max normalization pipeline described in Section III and enforcing Gradient Clipping in the optimizer. The final loss curve shows a smooth descent from 4.7 to 2.6.

D. Confusing Matrix Analysis

To further analyze classification errors, a Confusion Matrix was generated for a subset of 20 devices (Fig. 2). The matrix displays a prominent diagonal structure, which is a good representation of correct classifications.

However, notable off-diagonal clusters are observed between specific device groups (e.g., Device 12 and Device 18). These "confusion clusters" suggest that certain subsets of transmitters possess highly correlated hardware impairments, likely due to being manufactured in the same production batch or utilizing identical RF front-end components. This finding emphasizes the greatest challenge of RF fingerprinting, which is the degree of separation being limited by the manufacturing tolerances.

V. Analysis and Conclusion

This study presented a Deep Learning-based RF fingerprinting framework capable of identifying 150 distinct commercial WiFi devices using physical layer hardware impairments. This was done through the implementation of a robust DSP pipeline which features an I/Q decomposition, Min-Max normalization, and gradient clipping. Through this, the numerical instability inherent in raw USRP signal captures were successfully mitigated.

The experimental results confirm that manufacturing tolerances in RF front-end components act as a unique, non-forgeable signature for IoT authentication. The custom 1D-CNN architecture achieved a validation accuracy of 36.63% on a stratified 25% subset of the WiSig ManyTx dataset. This performance is approximately 55 times superior to random chance (0.66%). The confusion matrix analysis further revealed that misclassifications are primarily clustered among specific device groups, which likely indicates hardware batch correlations that present a fundamental physical limit to fingerprint distinctiveness.

While this study establishes a proof-of-concept for scalable device identification, several avenues remain for future optimization. First, due to computational resource limits, this study utilized a subset of the available data. Future work should leverage High-Performance Computing (HPC) clusters to train on the complete 100% signal set, which is expected to significantly enhance generalization and classification accuracy. Second, to further refine feature extraction, future iterations could explore Residual Networks (ResNets) [34] to deepen the model without gradient degradation, or Long Short-Term Memory (LSTM) networks [35] to better capture the long-term temporal dependencies of the preamble signal. The ultimate objective is real-time IoT security. An important next step is porting the trained model to edge computing platforms such as a Raspberry Pi or an NVIDIA Jetson Nano to evaluate inference latency and feasibility for live spoofing detection.

Lastly, To ensure long-term robustness, future studies should validate model performance across different days to account for component aging and environmental temperature drifts.

ACKNOWLEDGMENT

The authors would like to acknowledge the use of Large Language Models (Google Gemini) for assistance in debugging the Python scripts, optimizing memory usage, the training pipeline, refining the code structure, and for helping the authors understand the discussed topic at an accelerated rate.

REFERENCES

[1] K. Zhao and L. Ge, "A survey on the internet of things security," in *2013 Ninth International Conference on Computational Intelligence and Security*, 2013, pp. 663–667.

[2] I. Butun, P. Österberg, and H. Song, "Security of the internet of things: Vulnerabilities, attacks, and countermeasures," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 1, pp. 616–644, 2020.

[3] H. Jafari, O. Omotere, D. Adesina, H.-H. Wu, and L. Qian, "Iot devices fingerprinting using deep learning," in *MILCOM 2018 - 2018 IEEE Military Communications Conference (MILCOM)*, 2018, pp. 1–9.

[4] Q. Xu, R. Zheng, W. Saad, and Z. Han, "Device fingerprinting in wireless networks: Challenges and opportunities," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 1, pp. 94–104, 2016.

[5] O. Ureten and N. Serinken, "Wireless security through rf fingerprinting," *Canadian Journal of Electrical and Computer Engineering*, vol. 32, no. 1, pp. 27–33, 2007.

[6] C. Bertoncini, K. Rudd, B. Nousain, and M. Hinders, "Wavelet fingerprinting of radio-frequency identification (rfid) tags," *IEEE Transactions on Industrial Electronics*, vol. 59, no. 12, pp. 4843–4850, 2012.

[7] K. Merchant, S. Revay, G. Stantchev, and B. Nousain, "Deep learning for rf device fingerprinting in cognitive communication networks," *IEEE Journal of Selected Topics in Signal Processing*, vol. 12, no. 1, pp. 160–167, 2018.

[8] W. Wang and L. Gan, "Radio frequency fingerprinting improved by statistical noise reduction," *IEEE Transactions on Cognitive Communications and Networking*, vol. 8, no. 3, pp. 1444–1452, 2022.

[9] Q. Li and W. Trappe, "Detecting spoofing and anomalous traffic in wireless networks via forge-resistant relationships," *IEEE Transactions on Information Forensics and Security*, vol. 2, no. 4, pp. 793–808, 2007.

[10] K. Zeng, K. Govindan, and P. Mohapatra, "Non-cryptographic authentication and identification in wireless networks [security and privacy in emerging wireless networks]," *IEEE Wireless Communications*, vol. 17, no. 5, pp. 56–62, 2010.

[11] N. Soltanieh, Y. Norouzi, Y. Yang, and N. C. Karmakar, "A review of radio frequency fingerprinting techniques," *IEEE Journal of Radio Frequency Identification*, vol. 4, no. 3, pp. 222–233, 2020.

[12] A. Jagannath, J. Jagannath, and P. S. P. V. Kumar, "A comprehensive survey on radio frequency (rf) fingerprinting: Traditional approaches, deep learning, and open challenges," *Computer Networks*, vol. 219, p. 109455, 2022.

[13] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *arXiv preprint arXiv:1810.07906*, 2018.

[14] T. Jian, B. C. Rendon, E. Ojuba, N. Soltani, Z. Wang, K. Sankhe, A. Gritsenko, J. Dy, K. Chowdhury, and S. Ioannidis, "Deep learning for rf fingerprinting: A massive experimental study," *IEEE Internet of Things Magazine*, vol. 3, no. 1, pp. 50–57, 2020.

[15] K. Sankhe, M. Belgiovine, F. Zhou, S. Riyaz, S. Ioannidis, and K. Chowdhury, "Oracle: Optimized radio classification through convolutional neural networks," in *IEEE INFOCOM 2019-IEEE conference on computer communications*. IEEE, 2019, pp. 370–378.

[16] M. Valkama, A. Springer, and G. Hueber, "Digital signal processing for reducing the effects of rf imperfections in radio devices—an overview," in *Proceedings of 2010 IEEE International Symposium on Circuits and Systems*. IEEE, 2010, pp. 813–816.

[17] S. Abbas, M. Abu Talib, Q. Nasir, S. Idhis, M. Alaboudi, and A. Mohamed, "Radio frequency fingerprinting techniques for device identification: a survey," *International Journal of Information Security*, vol. 23, no. 2, pp. 1389–1427, 2024.

[18] Q. Jiang and J. Sha, "Rf fingerprinting identification in low snr scenarios for automatic identification system," *IEEE Transactions on Wireless Communications*, vol. 23, no. 3, pp. 2070–2081, 2024.

[19] M. Shantal, Z. Othman, and A. A. Bakar, "A novel approach for data feature weighting using correlation coefficients and min–max normalization," *Symmetry*, vol. 15, no. 12, p. 2185, 2023.

[20] J. Yu, A. Hu, G. Li, and L. Peng, "A robust rf fingerprinting approach using multisampling convolutional neural network," *IEEE Internet of Things Journal*, vol. 6, no. 4, pp. 6786–6799, 2019.

[21] N. Soltani, G. Reus-Muns, B. Salehi, J. Dy, S. Ioannidis, and K. Chowdhury, "Rf fingerprinting unmanned aerial vehicles with non-standard transmitter waveforms," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 12, pp. 15 518–15 531, 2020.

[22] M. Irfan, S. Sciancalepore, and G. Oligeri, "On the reliability of radio frequency fingerprinting," *arXiv preprint arXiv:2408.09179*, 2024.

[23] S. Hanna, S. Karunaratne, and D. Cabric, "Wisig: A large-scale wifi signal dataset for receiver and channel agnostic rf fingerprinting," *IEEE Access*, vol. 10, pp. 22 808–22 818, 2022.

[24] H. Ş. Demiroğlu, M. A. Awan, and A. Kara, "An overview of challenges to long-term sustainability and scalability of radio frequency fingerprinting," in *2024 6th International Conference on Communications, Signal Processing, and their Applications (ICCSPA)*, 2024, pp. 1–6.

[25] D. Nouichi, M. Abdelsalam, Q. Nasir, and S. Abbas, "Iot devices security using rf fingerprinting," in *2019 Advances in Science and Engineering Technology International Conferences (ASET)*, 2019, pp. 1–7.

[26] S. Al-Hazbi, A. Hussain, S. Sciancalepore, G. Oligeri, and P. Papadimitratos, "Radio frequency fingerprinting via deep learning: Challenges and opportunities," in *2024 International Wireless Communications and Mobile Computing (IWCMC)*, 2024, pp. 0824–0829.

[27] B. Razavi, *RF Microelectronics*, 2nd ed. Prentice Hall, 2011.

[28] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015, software available from tensorflow.org. [Online]. Available: https://www.tensorflow.org/

[29] F. Chollet *et al.*, "Keras," https://keras.io, 2015.

[30] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, http://www.deeplearningbook.org.

[31] L. Prechelt, "Early stopping - but when?" in *Neural Networks: Tricks of the Trade*. Springer, 1998, pp. 55–69.

[32] Y. Bengio, "Practical recommendations for gradient-based training of deep architectures," in *Neural Networks: Tricks of the Trade*. Springer, 2012, pp. 437–478.

[33] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[34] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[35] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

APPENDIX A
DETAILED PERFORMANCE METRICS

The following table presents the performance metrics across all 150 distinct classes. The model achieved an overall accuracy 36.63% test set. Given the high dimensionality of the problem (150 classes), this performance remains significantly above the random baseline of 0.66%, though it highlights specific classes (e.g., Classes 11, 29, 84) requiring further optimization to improve recall.

Due to the length of the report, the full class-by-class breakdown is presented below in raw text format.

	precision	recall	f1-score	support
0	0.3729	0.2444	0.2953	1207
1	0.5085	0.5222	0.5152	1440
2	0.3306	0.1023	0.1563	1202
3	0.4302	0.5035	0.4640	1414
4	0.3074	0.2667	0.2856	1440
5	0.3754	0.2084	0.2680	1142
6	0.4812	0.5345	0.5064	1435
7	0.4703	0.5660	0.5137	1440
8	0.3236	0.3138	0.3186	1380
9	0.1731	0.0382	0.0626	1177
10	0.2401	0.2562	0.2479	1440
11	0.0396	0.1073	0.0578	895
12	0.4649	0.3907	0.4246	1185
13	0.3142	0.2736	0.2925	1440
14	0.3530	0.1703	0.2298	1368
15	0.4041	0.4344	0.4187	1372
16	0.4514	0.4674	0.4592	1440
17	0.3948	0.5083	0.4444	1440
18	0.4155	0.5609	0.4774	1421
19	0.4153	0.2515	0.3133	1189
20	0.3322	0.3410	0.3365	1440
21	0.3562	0.3473	0.3517	1434
22	0.4420	0.5507	0.4904	1440
23	0.4213	0.3757	0.3972	1440
24	0.0586	0.3912	0.1020	749
25	0.3022	0.2956	0.2989	1424
26	0.3192	0.2573	0.2849	1434
27	0.3458	0.2779	0.3082	1360
28	0.3531	0.2812	0.3131	1440
29	0.0435	0.1232	0.0643	633
30	0.4184	0.3451	0.3782	1440
31	0.3310	0.4173	0.3692	1361
32	0.3927	0.3303	0.3588	1435
33	0.3892	0.2919	0.3336	1264
34	0.4224	0.5444	0.4757	1440
35	0.4630	0.4669	0.4649	1420
36	0.3120	0.2639	0.2859	1440
37	0.3742	0.3903	0.3821	1440
38	0.2650	0.2271	0.2446	1440
39	0.2910	0.4241	0.3452	1377
40	0.5466	0.4743	0.5079	1360
41	0.3366	0.3173	0.3267	1390
42	0.3263	0.5323	0.4046	1424
43	0.3168	0.2174	0.2578	1440
44	0.2512	0.3037	0.2749	1429
45	0.2923	0.1729	0.2173	1278
46	0.4175	0.3868	0.4016	1440
47	0.4741	0.4569	0.4653	1440
48	0.2697	0.2428	0.2555	1380
49	0.3392	0.3120	0.3250	1362
50	0.4837	0.4058	0.4413	1205
51	0.4339	0.6062	0.5058	1440
52	0.6431	0.4431	0.5247	1440
53	0.5219	0.2809	0.3652	1189
54	0.3358	0.2181	0.2644	1440
55	0.4523	0.6049	0.5175	1402
56	0.3132	0.1859	0.2333	1366
57	0.4427	0.5203	0.4783	1432
58	0.3554	0.2076	0.2621	1296
59	0.3294	0.2220	0.2652	1383
60	0.2449	0.3243	0.2790	1400
61	0.3710	0.4111	0.3900	1428
62	0.3982	0.2661	0.3191	1161
63	0.3403	0.6711	0.4516	1362
64	0.4019	0.2481	0.3068	1346
65	0.3539	0.3468	0.3503	1390
66	0.3522	0.3954	0.3725	1434
67	0.4754	0.4229	0.4476	1440
68	0.3300	0.3861	0.3558	1440
69	0.4917	0.3301	0.3950	1439
70	0.3225	0.3125	0.3174	1424
71	0.3407	0.2153	0.2638	1440
72	0.3330	0.2649	0.2951	1389
73	0.3011	0.2569	0.2773	1440
74	0.2520	0.2583	0.2551	1440
75	0.2920	0.2539	0.2716	1410
76	0.1524	0.1624	0.1573	1133
77	0.3686	0.2856	0.3218	1439
78	0.3709	0.3817	0.3763	1378
79	0.4203	0.3869	0.4029	1357
80	0.2647	0.2403	0.2519	1440
81	0.3213	0.3639	0.3413	1440
82	0.3551	0.3208	0.3371	1440
83	0.2406	0.2889	0.2625	1440
84	0.0332	0.1149	0.0515	818
85	0.2600	0.2715	0.2656	1440
86	0.2882	0.2838	0.2860	1434
87	0.3313	0.2414	0.2793	1363
88	0.3774	0.2993	0.3338	1440
89	0.3776	0.4761	0.4212	1111
90	0.6237	0.4606	0.5299	1281
91	0.4365	0.6814	0.5321	1331
92	0.4869	0.3492	0.4067	1011
93	0.4546	0.3688	0.4072	1440
94	0.3925	0.3903	0.3914	1440
95	0.4169	0.3361	0.3722	1440
96	0.4613	0.4275	0.4438	1380
97	0.5371	0.5674	0.5518	1440
98	0.4058	0.4549	0.4289	1440
99	0.3651	0.2637	0.3062	1278
100	0.3990	0.3278	0.3599	1440
101	0.3398	0.3174	0.3282	1440
102	0.5336	0.6174	0.5724	1440
103	0.3192	0.2517	0.2815	1434
104	0.4266	0.3147	0.3622	1303
105	0.4267	0.4951	0.4584	1440
106	0.2248	0.2500	0.2367	1400
107	0.3957	0.3045	0.3442	1271
108	0.4758	0.5931	0.5280	1440
109	0.3814	0.5250	0.4418	1440
110	0.6683	0.7736	0.7171	1440
111	0.4101	0.3438	0.3740	1440
112	0.2916	0.2794	0.2854	1360
113	0.4095	0.4590	0.4329	1440
114	0.4759	0.4447	0.4597	1419
115	0.3098	0.3023	0.3060	1204
116	0.2856	0.2787	0.2821	1360
117	0.3426	0.2486	0.2882	1287
118	0.4035	0.3674	0.3846	1440
119	0.3259	0.4600	0.3815	1437
120	0.4875	0.4965	0.4920	1416
121	0.5985	0.7135	0.6509	1431
122	0.4727	0.4282	0.4493	1434
123	0.5251	0.3000	0.3818	1360
124	0.3422	0.2460	0.2862	1362
125	0.3678	0.2424	0.2922	1440
126	0.4222	0.2618	0.3232	1440
127	0.4297	0.4861	0.4562	1440
128	0.3176	0.2331	0.2689	1360
129	0.3299	0.2694	0.2966	1440
130	0.4407	0.6451	0.5237	1440
131	0.4193	0.2601	0.3210	1138
132	0.2749	0.2045	0.2346	1281
133	0.4770	0.6331	0.5441	1360
134	0.4610	0.5386	0.4968	1439
135	0.3620	0.3042	0.3306	1440
136	0.3190	0.2342	0.2701	1200
137	0.3020	0.2404	0.2677	1281
138	0.2642	0.3965	0.3171	1440
139	0.3027	0.1994	0.2405	1439
140	0.4141	0.2430	0.3063	1280
141	0.3467	0.5806	0.4342	1440
142	0.6625	0.8139	0.7304	1440
143	0.5811	0.7486	0.6543	1440
144	0.4829	0.5367	0.5084	1157
145	0.5338	0.3534	0.4252	1364
146	0.2943	0.2604	0.2763	1440
147	0.3862	0.3498	0.3671	1072
148	0.3876	0.2520	0.3054	1369
149	0.3024	0.2523	0.2751	1280
accuracy			0.3663	204129
macro avg	0.3752	0.3616	0.3599	204129
weighted avg	0.3793	0.3663	0.3651	204129

APPENDIX B
EXPERIMENTAL CONFIGURATION

The specific hyperparameters and hardware environment used to obtain the reported results are detailed in Table II. To ensure reproducibility, the system was evaluated using the "Lite" training configuration to demonstrate feasibility on resource-constrained hardware.

TABLE II
HYPERPARAMETERS & HARDWARE SETUP

Parameter	**Value**
Data & Model Settings	
Input Shape	$(256, 2)$
Model Architecture	1D-CNN (3 Conv Blocks)
Optimizer	Adam
Learning Rate	1×10^{-5}
Gradient Clipping	`clipvalue = 0.5`
Batch Size	32 (Resource Constrained)
Dropout Rate	0.5
Epochs	20 (Early Stopping Enabled)
Environment	
CPU	Intel Core i7-13700KF
GPU	NVIDIA RTX 3060 (12GB)
RAM	16 GB DDR4
Software Stack	Python 3.10, TensorFlow 2.10